



CSC441

Foundation of Natural Language Processing

2/27/2026

Dr. Maryam Al-Berry

Mariam.csc0275@miuegypt.edu_1eg

Contents

- Sentence Segmentation
- Stemming
- Lemmatization
- Expanding Contractions
- Stopword Removal
- Exploratory Text Analysis
- N-Gram Language Models

Sentence Segmentation

Sentence Segmentation

- Tokenization is an instance of a more general problem of **segmentation**.
- Manipulating texts at the level of individual words often presupposes the ability to divide a text into individual sentences.
- Some corpora already provide access at the sentence level.

Sentence Segmentation

- Tokenization is an instance of a more general problem of **segmentation**.
- Manipulating texts at the level of individual words often presupposes the ability to divide a text into individual sentences.
- Some corpora already provide access at the sentence level.

Sentence Segmentation

- In other cases, the text is available only as a stream of characters.
- Before tokenizing the text into words, we need to segment it into sentences.

Stemming

Stemming

- Stemming is a text preprocessing task for transforming related or similar variants of a word (such as walking) to its base form (to walk), as they share the same meaning.
- One of the basic transformation stemming actions is to reduce a plural word to its singular form: apples is reduced to apple, for example.

Stemming

- In the shown example, stemming has reduced the different verb (enjoying, enjoy) and adjective (enjoyable) forms of a word to its base form, enjoy.

```
>>> import nltk
>>> from nltk.stem import PorterStemmer
>>> stemming = PorterStemmer()
>>> stemming.stem("enjoying")
'enjoy'
>>> stemming.stem("enjoys")
'enjoy'
>>> stemming.stem("enjoyable")
'enjoy'
```
- The Porter algorithm used by the stemmer utilizes various language-specific rules (in this case, English) to arrive at the stem words.
- One of these rules is removing suffixes such as ing from the word, as seen in the aforementioned example code.

Lemmatization

Lemmatization

- The process of lemmatization is very similar to stemming—you remove word affixes to get to a base form of the word.
- But in this case, this base form is also known as the root word, but not the root stem.
- The difference is that the root stem may not always be a lexicographically correct word; that is, it may not be present in the dictionary.
- The root word, also known as the **lemma**, will always be present in the dictionary.
- The lemmatization process is considerably slower than stemming because an additional step is involved where the root form or lemma is formed by removing the affix from the word if and only if the lemma is present in the dictionary.

Lemmatization

```
# lemmatize nouns
```

```
In [515]: print wnl.lemmatize('cars', 'n')
```

```
...: print wnl.lemmatize('men', 'n')
```

```
car
```

```
men
```

```
# lemmatize verbs
```

```
In [516]: print wnl.lemmatize('running', 'v')
```

```
...: print wnl.lemmatize('ate', 'v')
```

```
run
```

```
eat
```

```
# lemmatize adjectives
```

```
In [517]: print wnl.lemmatize('saddest', 'a')
```

```
...: print wnl.lemmatize('fancier', 'a')
```

```
sad
```

```
fancy
```

Expanding Contractions

Expanding Contractions

- Contractions are shortened version of words or syllables.
- They exist in either written or spoken forms.
- Shortened versions of existing words are created by removing specific letters and sounds.
- In case of English contractions, they are often created by removing one of the vowels from the word.
- **Examples** would be **is not** to **isn't** and **will not** to **won't**, where you can notice the apostrophe being used to denote the contraction and some of the vowels and other letters being removed.
- Usually contractions are avoided when used in formal writing, but informally, they are used quite extensively.

Expanding Contractions

- Various forms of contractions exist that are tied down to the type of auxiliary verbs that give us normal contractions, negated contractions, and other special conversational contractions, some of which may not involve auxiliaries.
- By nature, contractions do pose a problem for NLP and text analytics because, to start with, we have a special apostrophe character in the word.
- Plus we have two or more words represented by a contraction, and this opens a whole new can of worms when we try to tokenize this or even standardize the words.
- Hence, there should be some definite process by which we can deal with contractions when processing text.

Expanding Contractions

- Ideally, you can have a proper mapping for contractions and their corresponding expansions and then use it to expand all the contractions in your text.

```
CONTRACTION_MAP = {  
    "isn't": "is not",  
    "aren't": "are not",  
    "can't": "cannot",  
    "can't've": "cannot have",  
    .  
    .  
    .  
    "you'll've": "you will have",  
    "you're": "you are",  
    "you've": "you have"  
}
```


Stopword Removal

Stop Word Removal

- Commonly used words in English such as the, is, he, and so on, are generally called **stop words**.
- Other languages have similar commonly used words that fall under the same category.
- Stop word removal is another common preprocessing step for an NLP application.

Stop Word Removal

- In this step, we remove words that do not signify any importance to the document, such as grammar articles and pronouns.
- Some examples of such words are a, an, he, and her.
- By themselves, these words may not have an impact on NLP tasks, such as text categorization or search, as they are frequently used throughout the text.

Exploratory Text Analysis

Exploratory analysis of text

- Once we have the tokenized data, one of the basic analyses that is commonly performed is counting words or tokens and their distributions in the document.
- This will enable us to know more about the main topics in the document.

Exploratory Analysis of Text

- To get the frequency distribution of the words in the text, we can utilize the `nltk.FreqDist()` function, which obtains the top words used in the text, providing a rough idea

```
>>> frequency_dist = nltk.FreqDist(webtext_words)
>>> sorted(frequency_dist, key=frequency_dist.__getitem__,
reverse=True) [0:30]
['.', 'in', 'to', '"', 'the', "'", 'not', '-', 'when', 'on', 'a', 'is',
't', 'and', 'of', '(', 'page', 'for', 'with', ')', 'window', 'Firefox',
'does', 'from', 'open', ':', 'menu', 'should', 'bar', 'tab']
```

Exploratory Analysis of Text

- We can also look at the frequency distribution of words with a length greater than 3, which will exclude words such as **the** and **is**:

```
>>> large_words = dict([(k,v) for k,v in frequency_dist.items() if  
len(k)>3])  
>>> frequency_dist = nltk.FreqDist(large_words)  
>>> frequency_dist.plot(50,cumulative=False)
```

N-Gram Language Models

N-Gram Language Models

- A **language model (LM)** is a machine learning model that predicts upcoming words.
- More formally, a language model assigns a probability to each possible next word, or equivalently gives a probability distribution over possible next words.
- Language models can also assign a probability to an entire sentence.

N-Gram Language Models

- Why would we want to predict upcoming words?
- The main reason is that large language models are built just by training them to predict words!!
- Large Language Models (LLMs) learn an enormous amount about language solely from being trained to predict upcoming words from neighboring words.

N-Gram Language Models

- This probabilistic knowledge can be very practical.
- Consider correcting grammar or spelling errors like **Their are two midterms**, in which **There** was mistyped as **Their**, or **Everything has improve**, in which **improve** should have been **improved**.
- The phrase **There are** is more probable than **Their are**, and **has improved** than **has improve**, so a language model can help users select the more grammatical variant.

N-Gram Language Models

- An n-gram is a sequence of n words: a 2-gram (which we'll call **bigram**) is a two-word sequence of words like **The water**, or **water of**, and a 3-gram (a trigram) is a three-word sequence of words like **The water of**, or **water of Walden**.
- But we also 'ngram' to mean a **probabilistic model that can estimate the probability of a word given the n-1 previous words**, and thereby also to assign probabilities to entire sequences.

N-Gram Language Models

- Let's begin with the task of computing $P(w|h)$, the probability of a word w given some history h .
- Suppose the history h is “The water of Walden Pond is so beautifully ” and we want to know the probability that the next word is blue:

$P(\text{blue} | \text{The water of Walden Pond is so beautifully})$

- One way to estimate this probability is directly from relative frequency counts: take a very large corpus, count the number of times we see The water of Walden Pond is so beautifully, and count the number of times this is followed by blue.
- $$P(\text{blue} | \text{The water of Walden Pond is so beautifully}) = \frac{C(\text{The water of Walden Pond is so beautifully blue})}{C(\text{The water of Walden Pond is so beautifully})}$$

N-Gram Language Models

- If we had a large enough corpus, we could compute these two counts and estimate the probability.
- But even the entire web isn't big enough to give us good estimates for counts of entire sentences.
- This is because language is creative; new sentences are invented all the time, and we can't expect to get accurate counts for such large objects as entire sentences.
- For this reason, we'll need more clever ways to estimate the probability of a word w given a history h , or the probability of an entire word sequence W .

N-Gram Language Models

- Now, how can we compute probabilities of entire sequences like $P(w_1; w_2; \dots; w_n)$?
- One thing we can do is decompose this probability using the chain rule of probability

$$\begin{aligned} P(X_1 \dots X_n) &= P(X_1)P(X_2|X_1)P(X_3|X_{1:2}) \dots P(X_n|X_{1:n-1}) \\ &= \prod_{k=1}^n P(X_k|X_{1:k-1}) \end{aligned}$$

Applying the chain rule to words, we get

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^n P(w_k|w_{1:k-1}) \end{aligned}$$

N-Gram Language Models

- The chain rule shows the link between computing the joint probability of a sequence and computing the conditional probability of a word given previous words.
- But using the chain rule doesn't really seem to help us! We don't know any way to compute the exact probability of a word given a long sequence of preceding words, $P(w_n | w_{1:n-1})$.

Markov Assumption

- The intuition of the n-gram model is that instead of computing the probability of a word given its entire history, we can approximate the history by just the last few words.
- The **bigram** model, for example, approximates the probability of a word given all the previous words $P(w_n | w_{1:n-1})$ by using only the conditional probability given the preceding word $P(w_n | w_{n-1})$.
- In other words, instead of computing the probability $P(\text{blue} | \text{The water of Walden Pond is so beautifully})$ we approximate it with the probability $P(\text{blue} | \text{beautifully})$.

Markov Assumption

- When we use a bigram model to predict the conditional probability of the next word, we are thus making the following approximation:

$$P(w_n | w_{1:n-1}) \approx P(w_n | w_{n-1})$$

- The assumption that the probability of a word depends only on the previous word is called a **Markov assumption**.
- Markov models are the class of probabilistic models that assume we can predict the probability of some future unit without looking too far into the past.

How to compute probabilities

- How do we estimate these bigram or n-gram probabilities?
- An intuitive way to estimate probabilities is called maximum likelihood estimation or MLE.
- We get the MLE estimate for the parameters of an n-gram model by getting counts from a corpus, and normalizing the counts so that they lie between 0 and 1.
- For probabilistic models, normalizing means dividing by some total count so that the resulting probabilities fall between 0 and 1 and sum to 1.

How to compute probabilities

- For example, to compute a particular bigram probability of a word w_n given a previous word w_{n-1} , we'll compute the count of the bigram $C(w_{n-1} w_n)$ and normalize by the sum of all the bigrams that share the same first word w_{n-1} :

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1} w_n)}{\sum_w C(w_{n-1} w)}$$

- We can simplify this equation, since the sum of all bigram counts that start with a given word w_{n-1} must be equal to the unigram count for that word w_{n-1}

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1} w_n)}{C(w_{n-1})}$$



Relative
Frequency

How to compute probabilities

- Example:

`<s> I am Sam </s>`

`<s> Sam I am </s>`

`<s> I do not like green eggs and ham </s>`

$$P(I | <s>) = \frac{2}{3} = 0.67 \quad P(\text{Sam} | <s>) = \frac{1}{3} = 0.33 \quad P(\text{am} | I) = \frac{2}{3} = 0.67$$

$$P(</s> | \text{Sam}) = \frac{1}{2} = 0.5 \quad P(\text{Sam} | \text{am}) = \frac{1}{2} = 0.5 \quad P(\text{do} | I) = \frac{1}{3} = 0.33$$

How to compute probabilities

- Example:

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

How to compute probabilities

- Example:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

↓

$$P(i | \langle s \rangle) = 0.25$$

$$P(\text{english} | \text{want}) = 0.0011$$

$$P(\text{food} | \text{english}) = 0.5$$

$$P(\langle /s \rangle | \text{food}) = 0.68$$

How to compute probabilities

- Example:

$$P(i | \langle s \rangle) = 0.25 \qquad P(\text{english} | \text{want}) = 0.0011$$

$$P(\text{food} | \text{english}) = 0.5 \qquad P(\langle /s \rangle | \text{food}) = 0.68$$

$$P(\langle s \rangle \text{ I want english food } \langle /s \rangle) =$$

$$P(I | \langle s \rangle)$$

$$\times P(\text{want} | I)$$

$$\times P(\text{english} | \text{want})$$

$$\times P(\text{food} | \text{english})$$

$$\times P(\langle /s \rangle | \text{food})$$

$$= .000031$$

What kinds of knowledge do N-grams represent?

- $P(\text{to}|\text{want}) = .66$
- $P(\text{want} | \text{spend}) = 0$
- $P(\text{of} | \text{to}) = 0$

- $P(\text{dinner}|\text{lunch or}) = .83$
- $P(\text{dinner}|\text{for}) \sim P(\text{lunch}|\text{for})$

- $P(\text{chinese}|\text{want}) > P(\text{english}|\text{want})$

Knowledge of grammar

("want" is followed by infinitive "to")

("of" is not a verb)

Knowledge of meaning

The words "dinner" or "lunch" are semantically related.

Knowledge about the world

Chinese food is very popular

Dealing with scale in large n-grams

- LM probabilities are stored and computed in log format, i.e. log probabilities
- This avoids underflow from multiplying many small numbers

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

- If we need probabilities we can do one exp at the end

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

Evaluation and Preplexity

How to evaluate N-gram models

"Extrinsic (in-vivo) Evaluation"

- To compare models A and B:
 1. Put each model in a real task
- Machine Translation, speech recognition, etc.
 2. Run the task, get a score for A and for B
- How many words translated correctly
- How many words transcribed correctly
- 2. Compare accuracy for A and B

How to evaluate N-gram models

- Extrinsic evaluation not always possible
- Expensive, time-consuming
- Doesn't always generalize to other applications

"Intrinsic evaluation: perplexity"

- Directly measures language model performance at predicting words.
- Doesn't necessarily correspond with real application performance
- But gives us a single general metric for language models
- Useful for large language models (LLMs) as well as n-grams

Training sets and Test sets

- We train parameters of our model on a **training set**.
- We test the model's performance on data we haven't seen.
 - A **test set** is an unseen dataset; different from training set.
 - Intuition: we want to measure generalization to unseen data
 - An **evaluation metric** (like **perplexity**) tells us how well our model does on the test set.

Choosing the training and test sets

- If we're building an LM for a specific task
 - The test set should reflect the task language we want to use the model for
- If we're building a general-purpose model
 - We'll need lots of different kinds of training data
 - We don't want the training set or the test set to be just from one domain or author or language.

Training on test data

We can't allow test sentences into the training set

- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called “Training on the test set”

Bad science!

Dev sets

- If we test on the test set many times we might implicitly tune to its characteristics
 - Noticing which changes make the model better.
- So we run on the test set only once, or a few times
- That means we need a third dataset:
 - A **development test set** or, **devset**.
 - We test our LM on the devset until the very end
 - And then test our LM on the **test set** once

Intuition of perplexity as evaluation metric

How good is our language model?

Intuition: A good LM prefers "real" sentences

- Assign higher probability to “real” or “frequently observed” sentences
- Assigns lower probability to “word salad” or “rarely observed” sentences?

Intuition of perplexity 2: Predicting Next Word



Claude Shannon

The Shannon Game: **How well can we predict the next word?**

- Once upon a _____
- That is a picture of a _____
- For breakfast I ate my usual _____

Unigrams are terrible at this game
(Why?)

time	0.9
dream	0.03
midnight	0.02
...	
and	1e-100

A good LM is one that assigns a higher probability to the next word that actually occurs

Intuition of perplexity 3: The best language model is one that best predicts the entire unseen test set

- We said: a good LM is one that assigns a higher probability to the next word that actually occurs.
- Let's generalize to all the words!
 - The best LM assigns high probability to the entire test set.
- When comparing two LMs, A and B
 - We compute $P_A(\text{test set})$ and $P_B(\text{test set})$
 - The better LM will give a higher probability to (=be less surprised by) the test set than the other LM.

Intuition of perplexity 4: Use perplexity instead of raw probability

- Probability depends on size of test set
 - Probability gets smaller the longer the text
 - Better: a metric that is **per-word**, normalized by length
- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Intuition of perplexity 5: Weighted Average Branching Factor

Perplexity is also the **weighted average branching factor** of a language.

Branching factor: number of possible next words that can follow any word

Example: Deterministic language $L = \{\text{red}, \text{blue}, \text{green}\}$

Branching factor = 3 (any word can be followed by red, blue, green)

Now assume LM A where each word follows any other word with equal probability $\frac{1}{3}$

Given a test set $T = \text{"red red red red blue"}$

$$\text{Perplexity}_A(T) = P_A(\text{red red red red blue})^{-1/5} = ((\frac{1}{3})^5)^{-1/5} = (\frac{1}{3})^{-1} = 3$$

- But now suppose red was very likely in training set, such that for LM B:
 - $P(\text{red}) = .8$ $p(\text{green}) = .1$ $p(\text{blue}) = .1$
- We would expect the probability to be higher, and hence the perplexity to be smaller:

$$\text{Perplexity}_B(T) = P_B(\text{red red red red blue})^{-1/5}$$

$$= (.8 * .8 * .8 * .8 * .1)^{-1/5} = .04096^{-1/5} = .527^{-1} = 1.89$$

Holding test set constant: Lower Perplexity. Better Language Model

- Training 38 million words, test 1.5 million words, WSJ

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

